

ECE3622

Embedded System Design

Han Wang

Electrical and Computer Engineering

Temple University

About Me:

- Assistant Professor in ECE:
 - Website: <https://sites.google.com/view/wanghanjane/home>
 - Contact: han.wang.hw@temple.edu
 - Office: 722 College of Engineering
- Research and Teaching:
 - Embedded System
 - Applied Machine Learning
 - Computer Architecture
 - Hardware Security

Logistics:

- Labs: ECE3623
- Every Tuesday/Thursday 2:00pm ~ 3:20pm
- Office hour: Tuesday 12:00pm ~ 1:00pm
- Textbook: digital version available on Canvas
 - *The Zynq Book*,
 - *The Zynq Book Tutorial*,
 - *The FreeRTOS Real-time Kernel*
- TA:
 - Jianlin Liu, jialinliu@temple.edu

Grading:

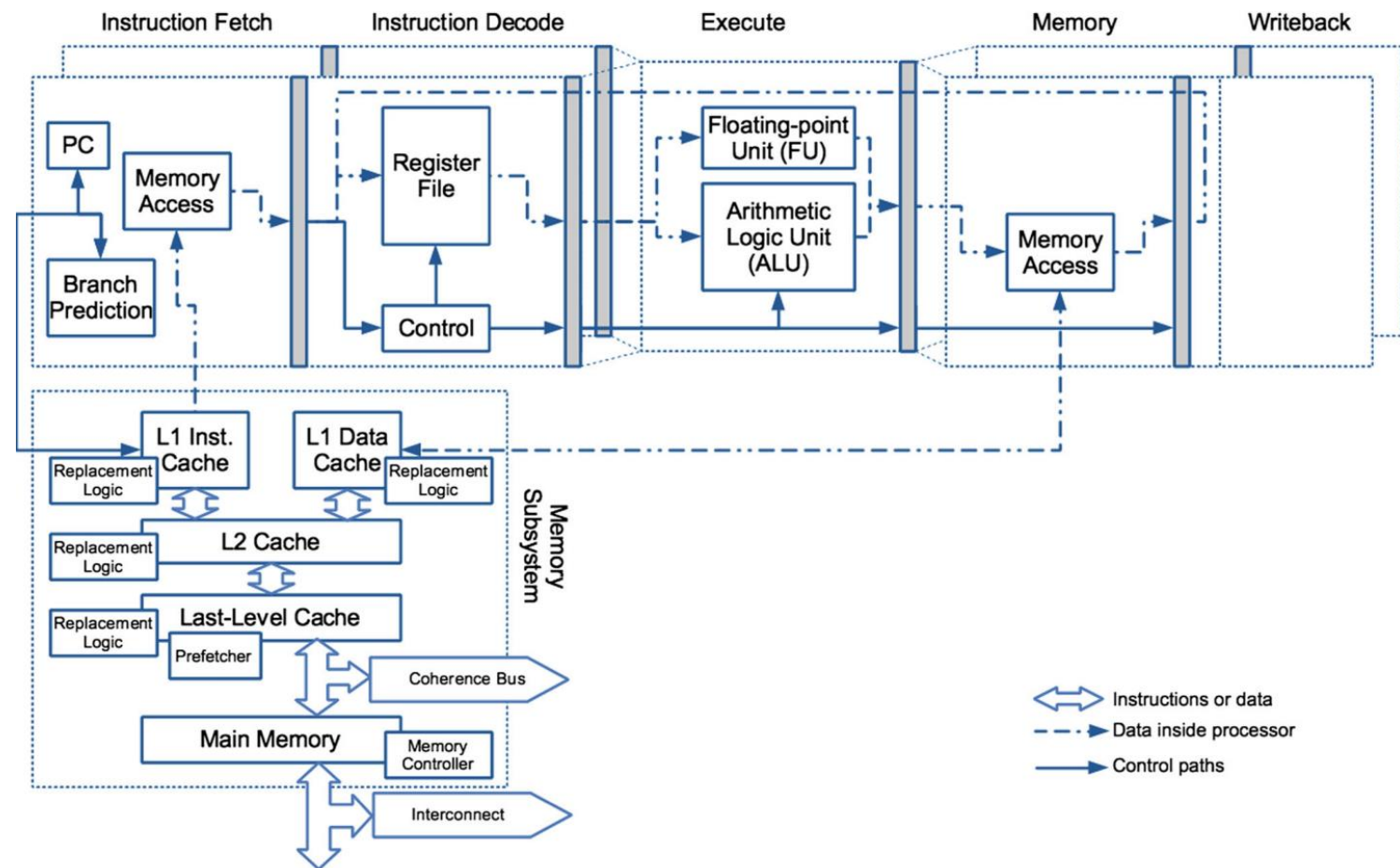
- Four Homework assignments: 20% (tentative, may change to in-class quiz).
- Three in-person exams ($15\% \times 3$) and one final exam (35%)

Agenda

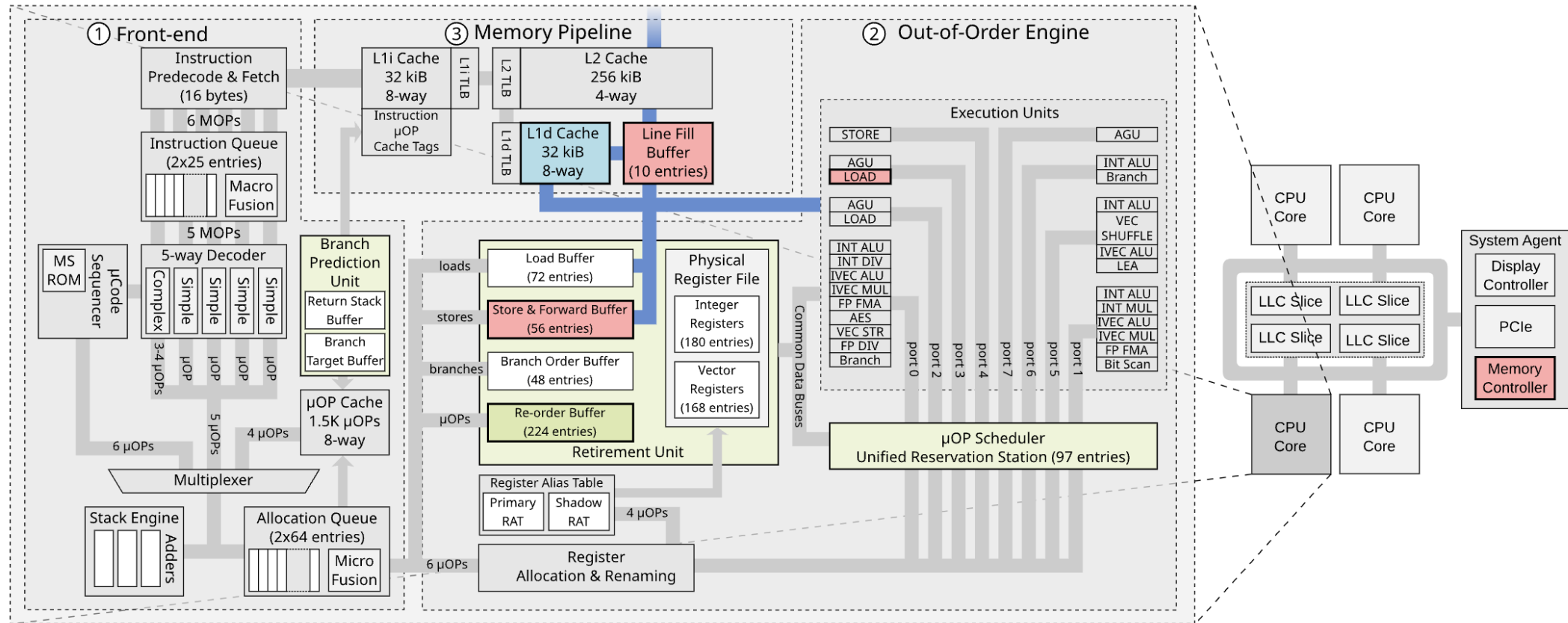
- Review Computer Architecture
- Embedded System and FPGA
- Embedded SoC
- Zynq Device:
 - Processing System
 - Programmable Logic
 - Interfaces
- Device Comparison: software core vs. hard core
- Design with Zynq
- Example FPGA-accelerated Addition

Overview of Computer Architecture

- Cache
- Branch
- Pipeline
- Execution Unit

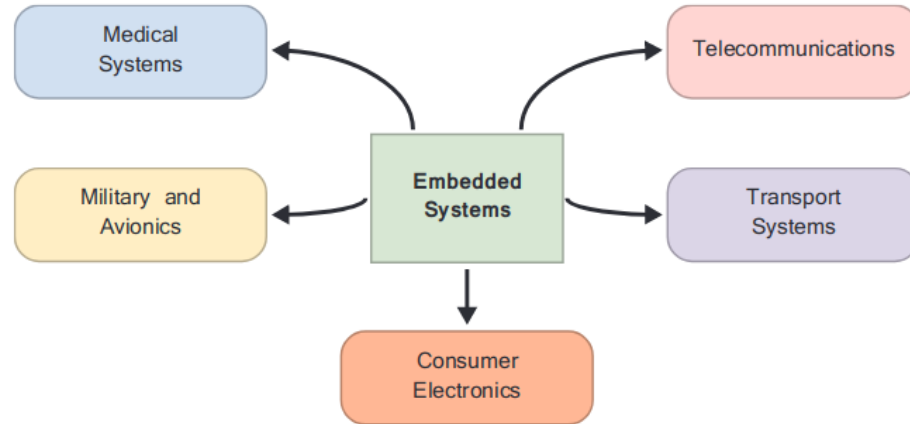


Overview of Computer Architecture



Embedded System

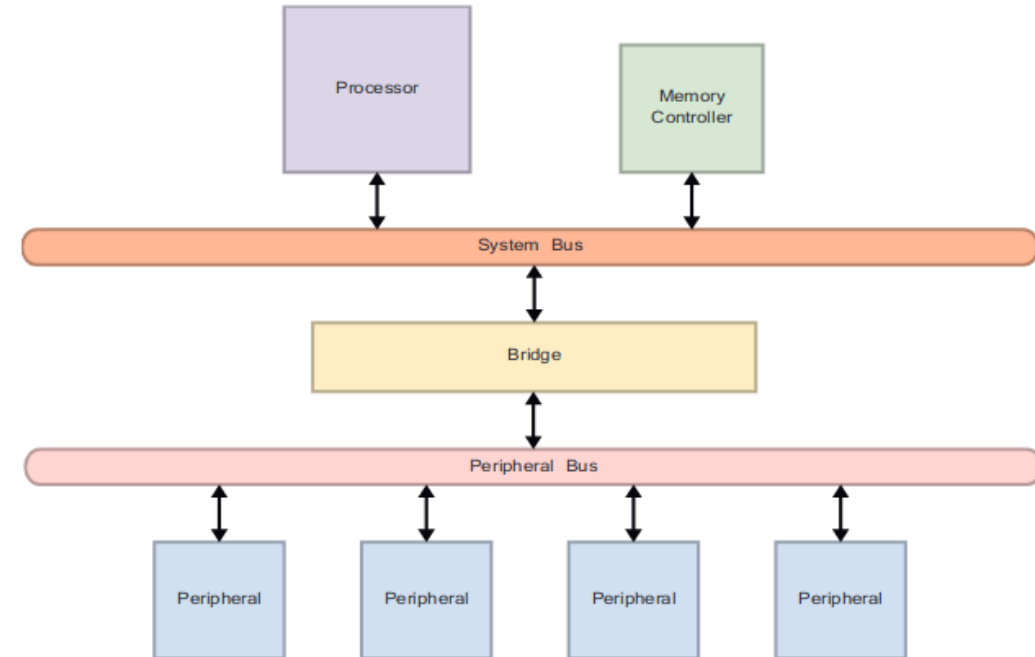
- Specialized computing system for **one** or **very few** dedicated functions.
- Deployed in a large number of devices across a breadth of fields.



- Constraints:
 - available system-memory
 - available processor speed
 - the need to limit power dissipation

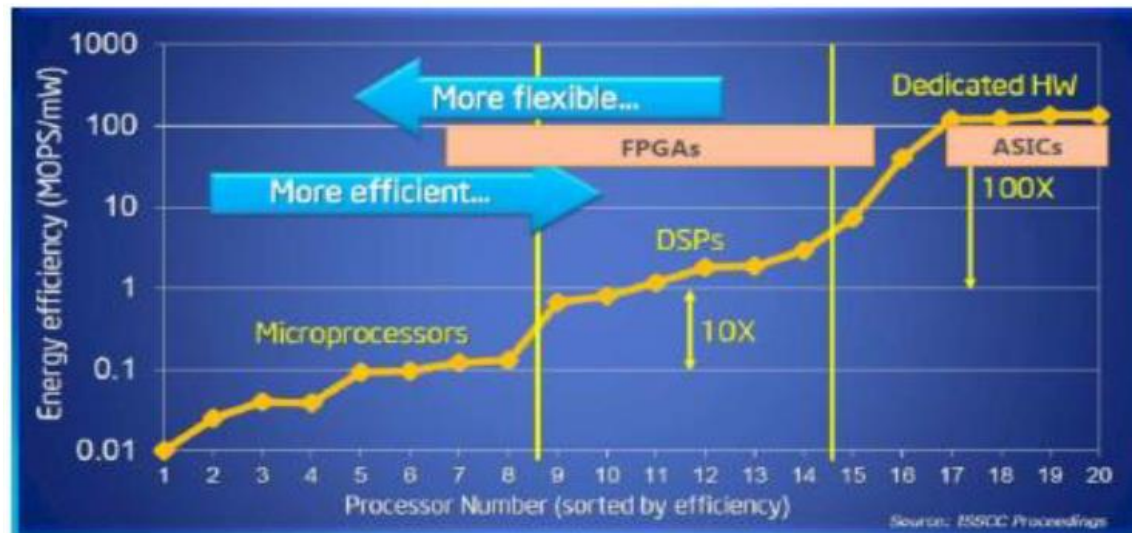
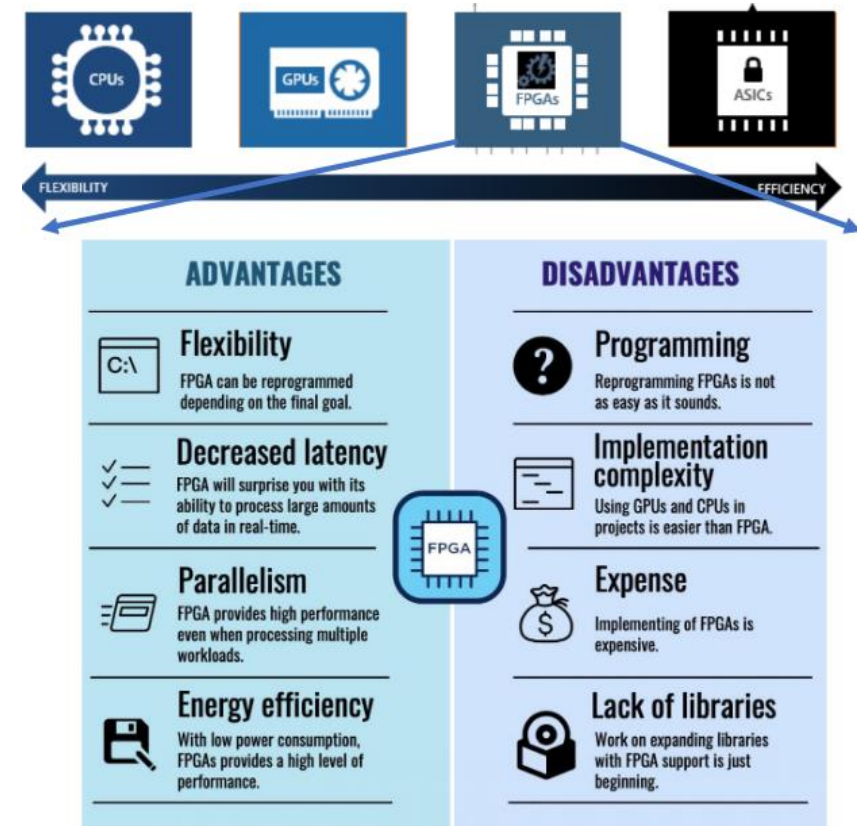
Embedded System

- Generic embedded system architecture:
 - Processor
 - Memory Controller
 - Peripherals
 - System Bus
 - Peripheral Bus
- Embedded processor:
 - Micro-controller: TI CC3200, ...
 - FPGA with System-on-chip (SoC): Zynq series



FPGA (Field Programmable Gate Arrays)

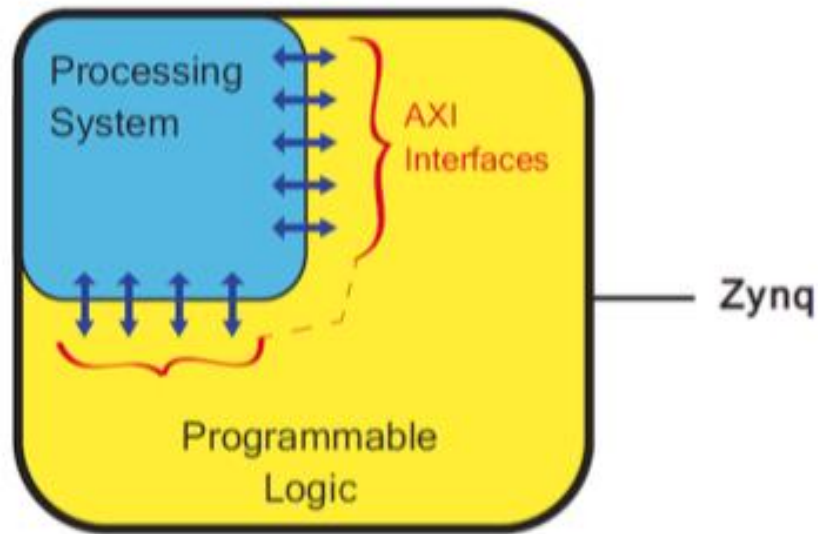
- Digital chips that can be programmed for implementing arbitrary digital circuits.
- Why FPGA?
 - Power-efficiency
 - Low-latency
 - Flexibility
 - Efficient Data Movement



Source: Bob Broderson, Berkeley Wireless group

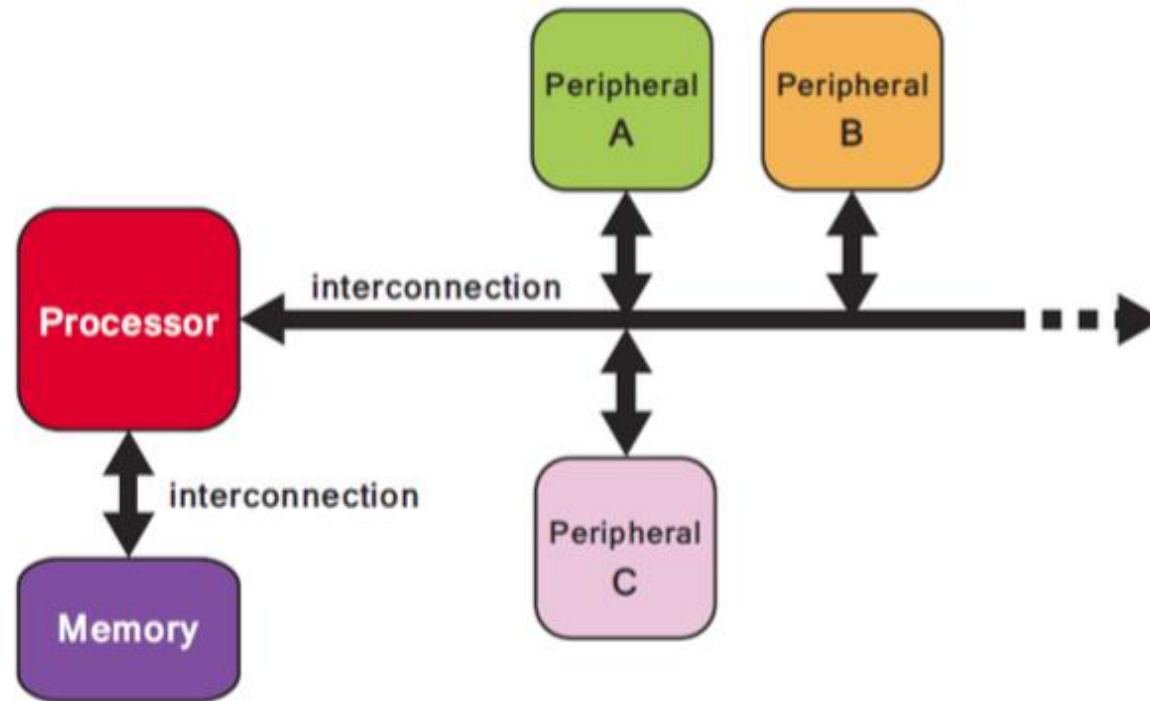
Embedded System-on-chip (SoC) Architecture

- Comprises two main parts: a processing system formed around a dual-core ARM Cortex A9 processor, and programmable logic (PL) which is equivalent to that of an FPGA.
- Integrated memory, a variety of peripherals and high speed communications interfaces.



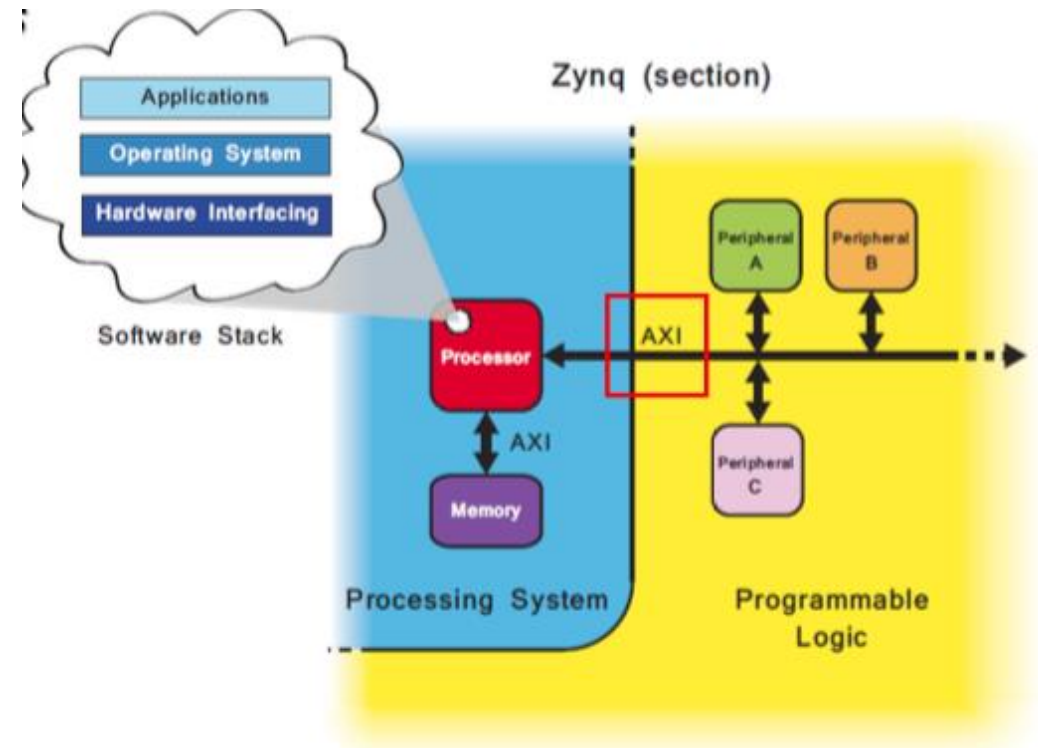
Embedded System-on-chip (SoC) Architecture

- The processor is the central element of the hardware system
- The software system is run on the processor with applications (usually based on operating system) and with a lower layer of software functionality with the hardware system.



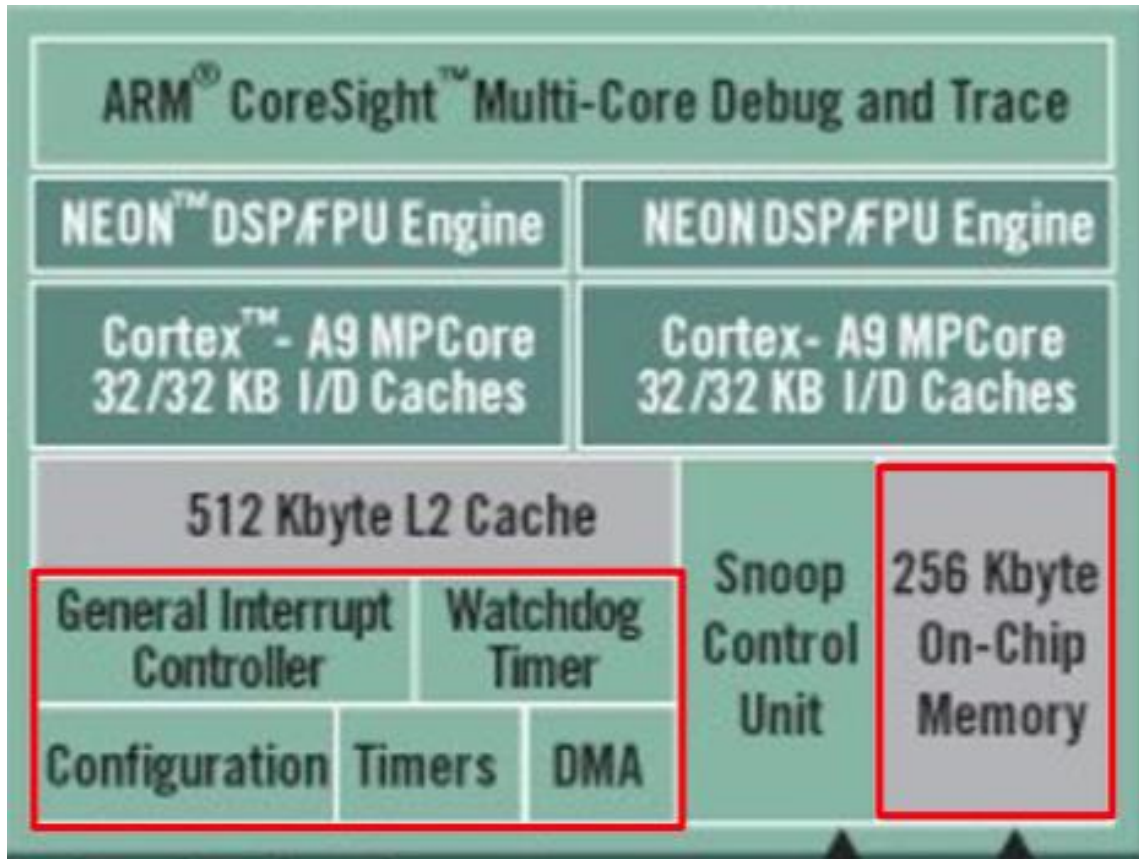
Embedded System-on-chip (SoC) Architecture

- The PS has a fixed architecture and hosts the processor and system memory.
- The PL is completely flexible and is used to create peripherals.
- The interconnections are implemented via Advanced eXtensible Interface (AXI).

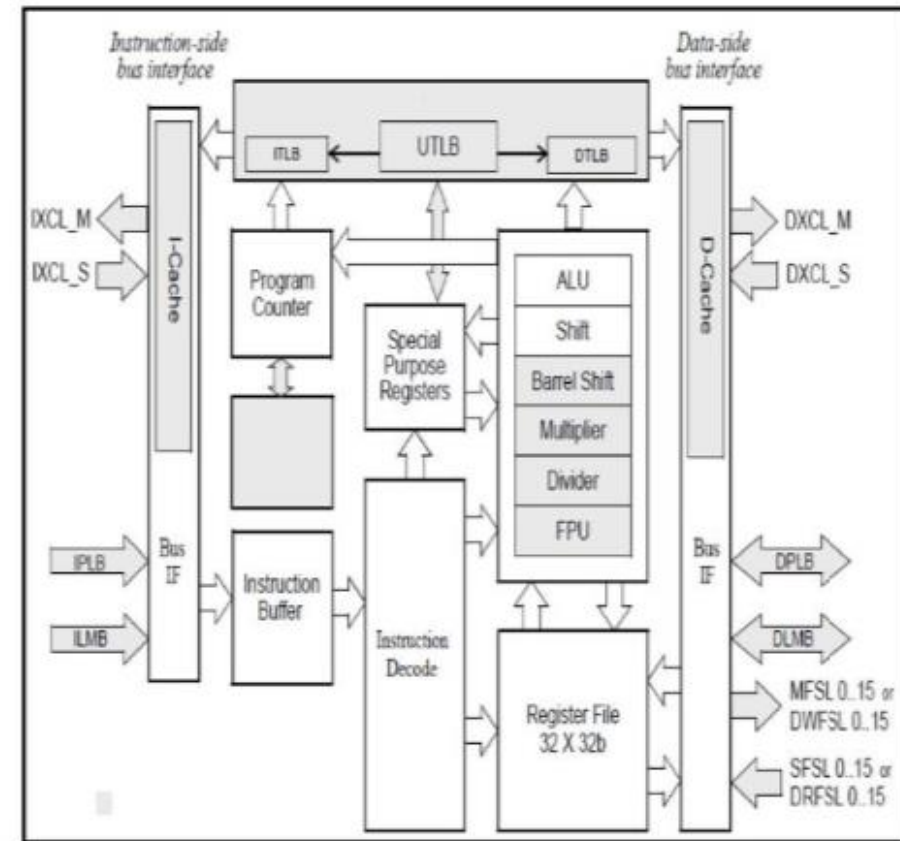


Zynq Devices: hard core vs soft core

- Significant differences include dual cores, timers, DMA, interrupt controller and 256 Kb on chip memory



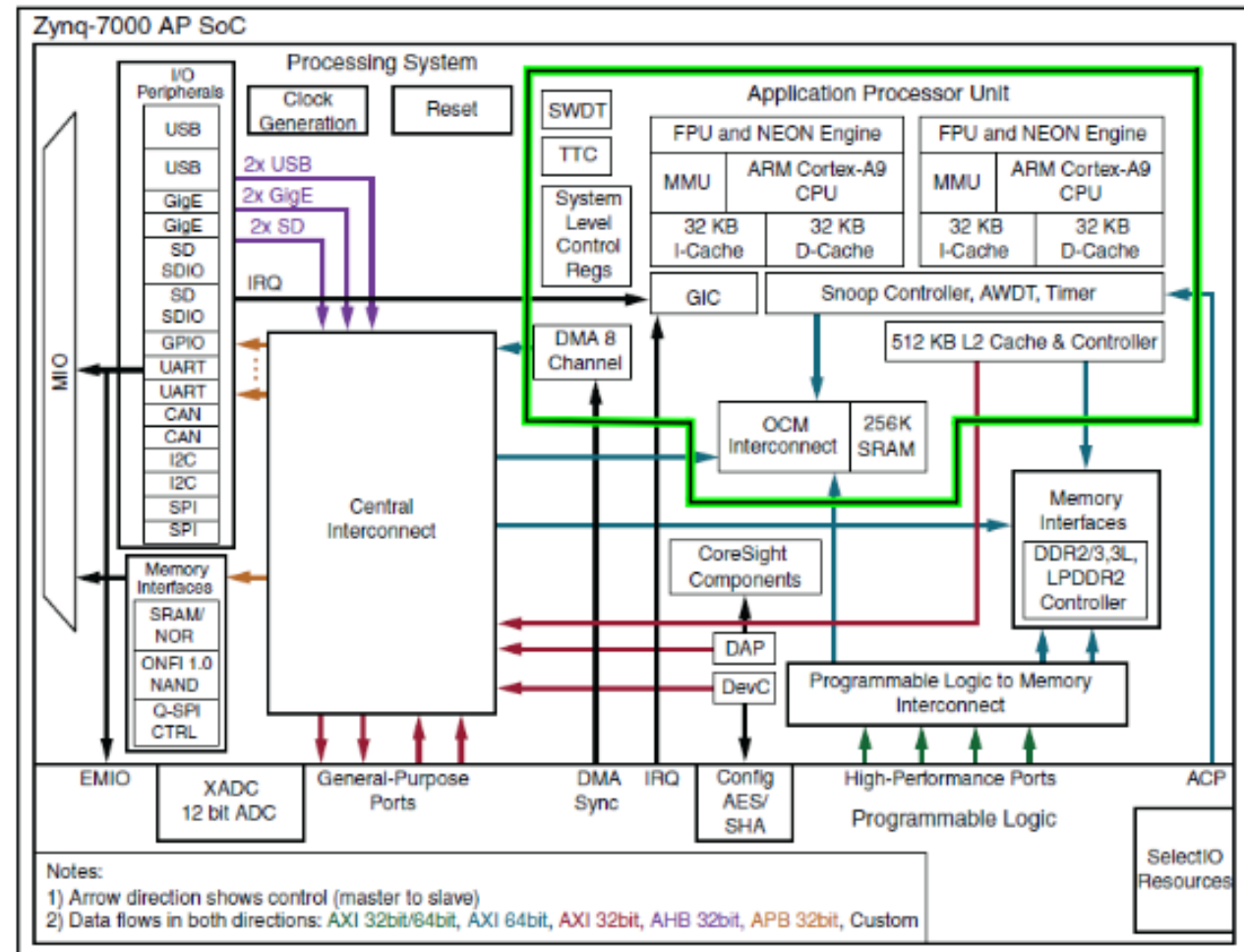
Zynq dual core architecture



Microblaze Architecture

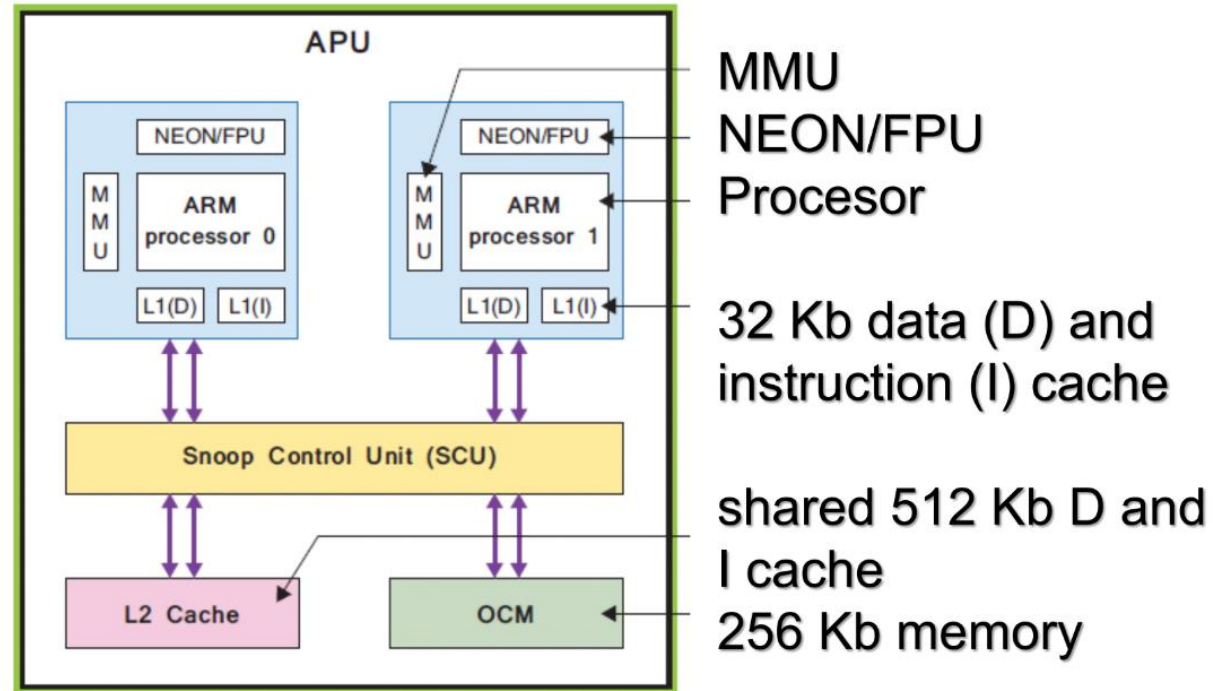
Zynq-7000 from Xilinx

- Processing systems is a dual-core ARM Cortex-A9 processor
- Programmable logic is based on Xilinx 7-series FPGA architecture
- Standard AXI interfaces



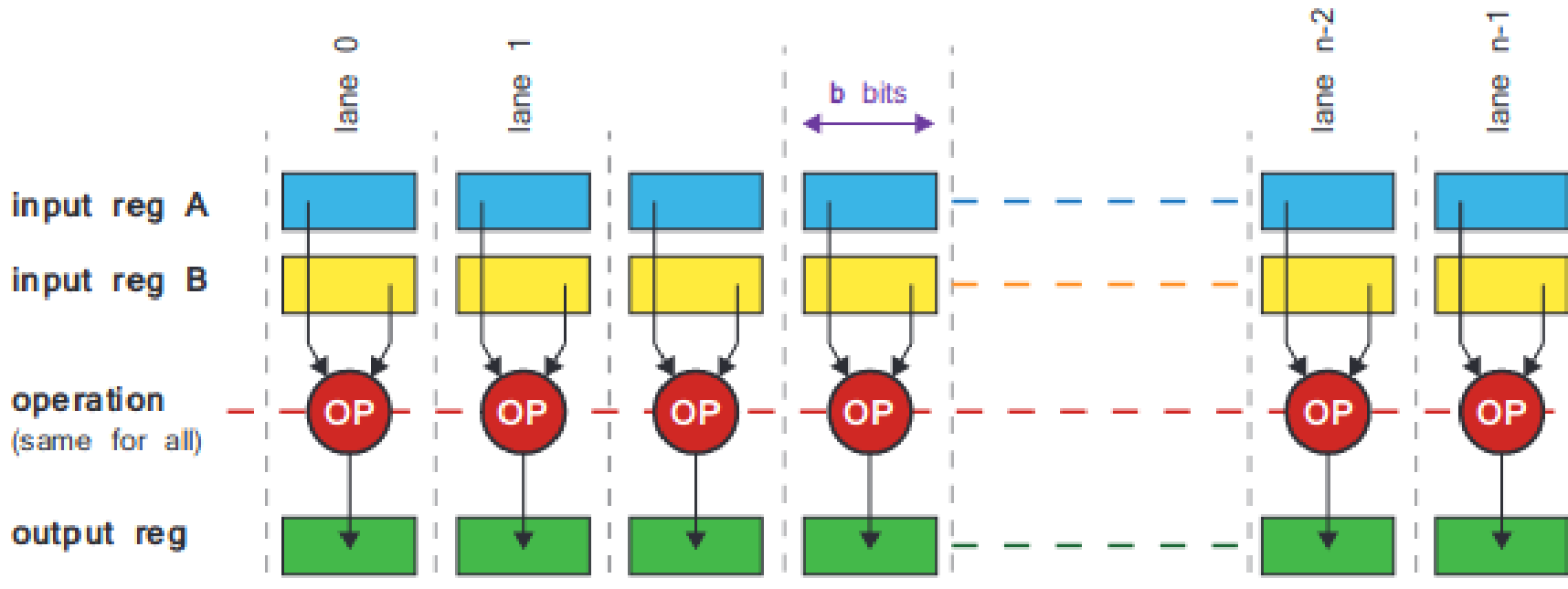
Application Processing Unit (APU)

- Consists of a Memory Management Unit (MMU), Graphical and Floating Point Unit (NEON/FPU) and memory



NEON

- The NEON engine provides Single Instruction Multiple Data (SIMD) facilities to enable acceleration of graphical media and DSP type algorithms.
- NEON accepts multiple sets of input vectors for the same operation performed simultaneously to provide a corresponding set of output vectors



The Floating Point Unit (FPU)

- The Floating Point Unit (FPU) provides hardware acceleration of floating point operations in compliance with the 32 or 64 bit IEEE 754 standard.

	Denormalized	Normalized	Approximate Decimal
32 bit Single Precision	$\pm 2^{-149}$ to $(1-2^{-23}) \times 2^{-126}$	$\pm 2^{-126}$ to $(2-2^{-23}) \times 2^{127}$	$\pm \approx 10^{-44.85}$ to $\approx 10^{38.53}$
64 bit Double Precision	$\pm 2^{-1074}$ to $(1-2^{-52}) \times 2^{-1022}$	$\pm 2^{-1022}$ to $(2-2^{-52}) \times 2^{1023}$	$\pm \approx 10^{-323.3}$ to $\approx 10^{308.3}$

32 bit FP Conversion

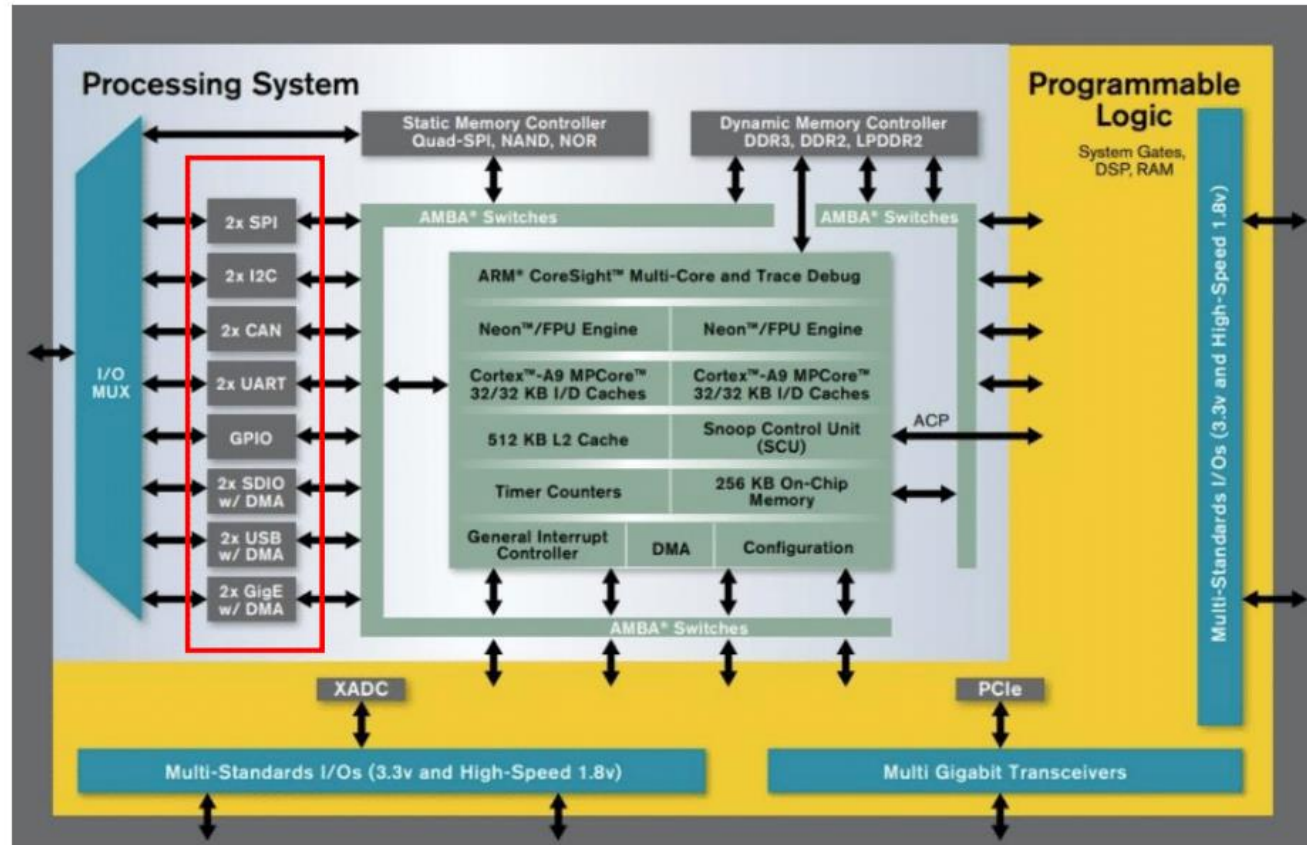
IEEE 754 Converter (JavaScript), V0.22

	Sign	Exponent	Mantissa
Value:	+1	2^6	1.9290000200271606
Encoded as:	0	133	7793017
Binary:	☐	☒ ☐ ☐ ☐ ☐ ☐ ☒ ☐ ☒	☒ ☒ ☒ ☐ ☒ ☒ ☐ ☒ ☒ ☐ ☒ ☐ ☐ ☒ ☐ ☐ ☒ ☒ ☒ ☐ ☐ ☐ ☒
You entered	123.456		
Value actually stored in float:	123.45600128173828125		
Error due to conversion:	0.00000128173828125		
Binary Representation	01000010111101101110100101111001		
Hexadecimal Representation	0x42f6e979		

+1
-1

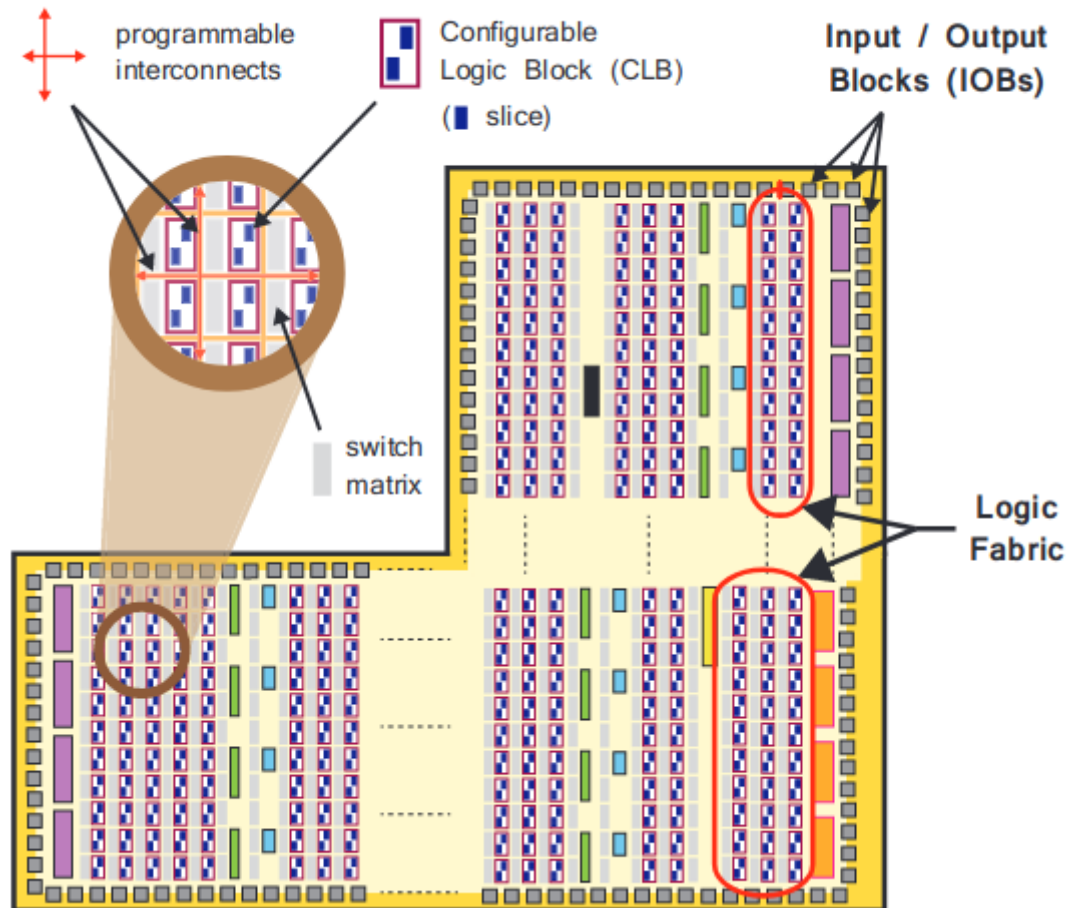
External Interfaces

- Communication between the PS and external interfaces is achieved primarily via the Multiplexed Input/Output (MIO), which provides 54 pins of flexible connectivity, meaning that the mapping between peripherals and pins can be defined as required.



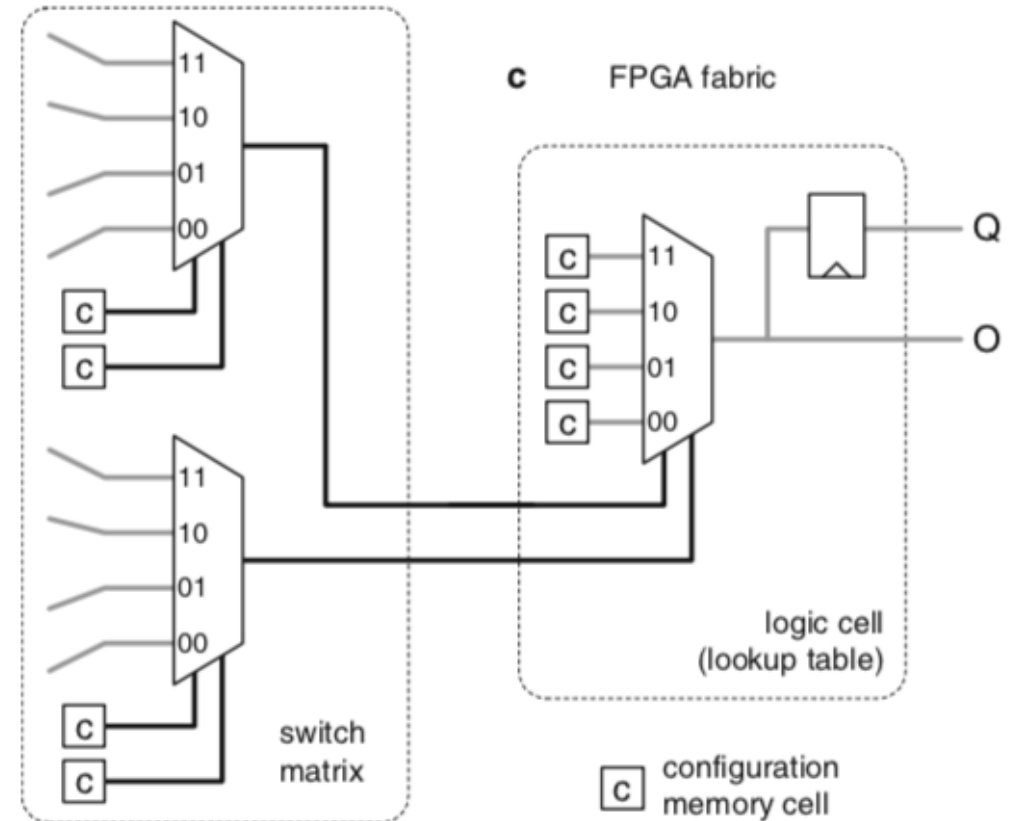
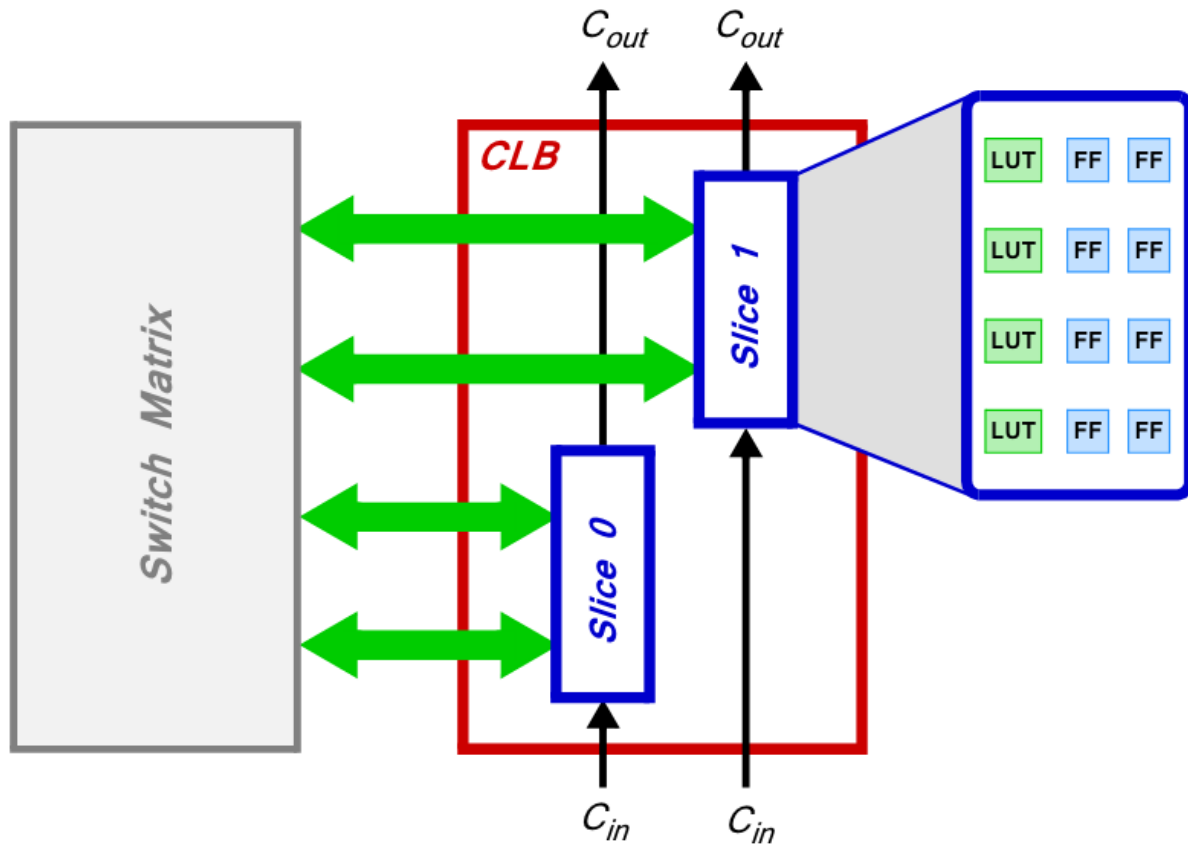
Programmable Logic (PL)

- is an FPGA logic fabric composed of slices and Configurable Logic Blocks (CLBs), and Input/Output Blocks (IOBs) for interfacing.



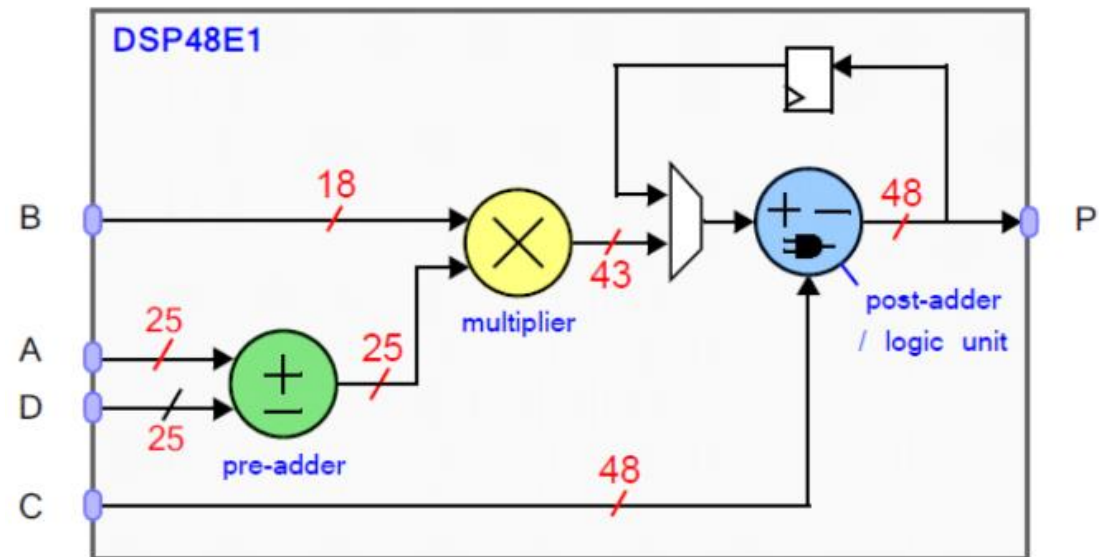
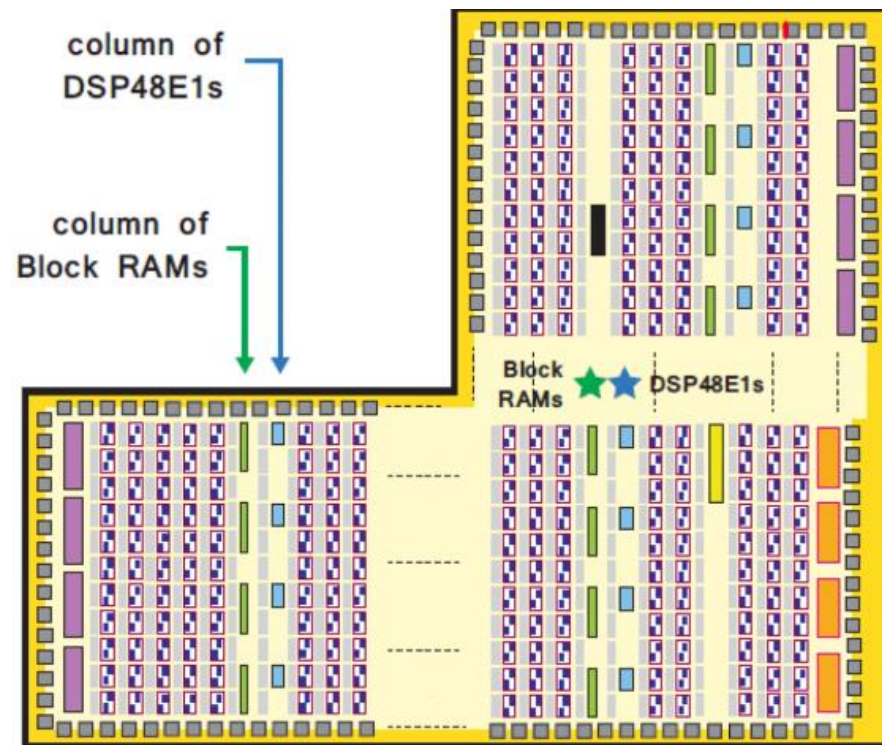
Programmable Logic (PL)

- Configurable Logic Block (CLB) — are small, regular groupings of logic elements that are laid out in a two-dimensional array on the PL, and connected to other similar resources via programmable interconnects.



Programmable Logic (PL)

- Block RAMs provide memory requirements
- DSP48E1 slices are hardware multipliers for high-speed arithmetic.



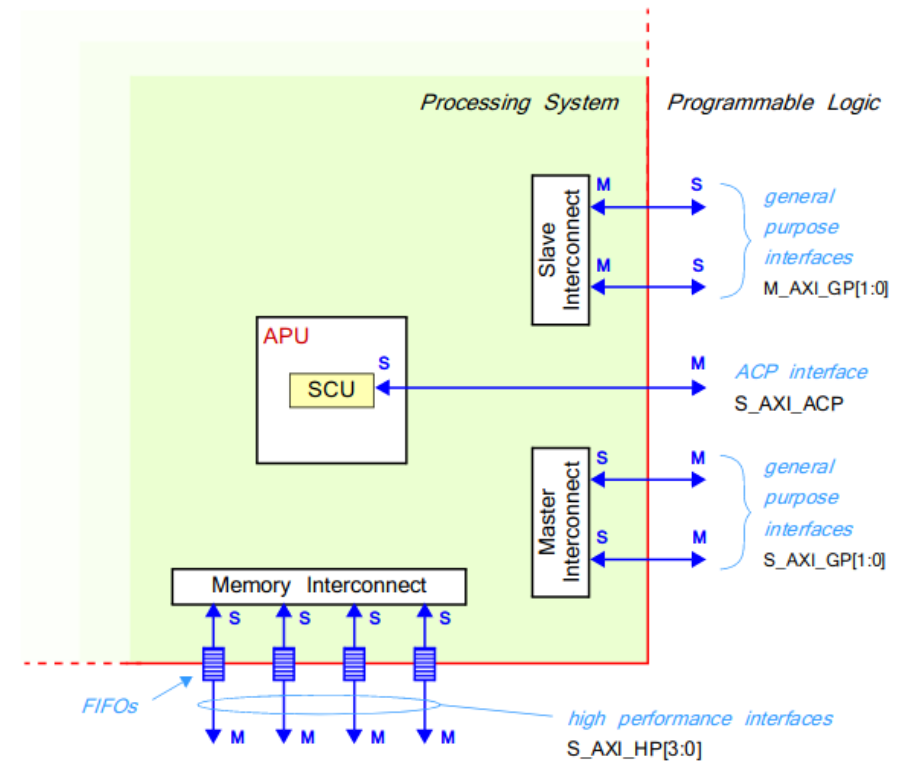
Primary Interface between the PS and PL: AXI

- The Advanced eXtensible Interface (AXI) standard current version is AXI4 and is part of the ARM AMBA® open standard.
- The AXI4 standard has three implementations: AXI4, AXI4-Lite and AXI4-Stream

	AXI4	AXI4-Lite	AXI4-Stream
Dedicated for	high-performance and memory mapped systems	register-style interfaces (area efficient implementation)	non-address based IP (PCIe, Filters, etc.)
Burst (data beta)	up to 256	1	Unlimited
Data width	32 to 1024 bits	32 or 64 bits	any number of bytes
Applications (examples)	Embedded, memory	Small footprint control logic	DSP, video, communication

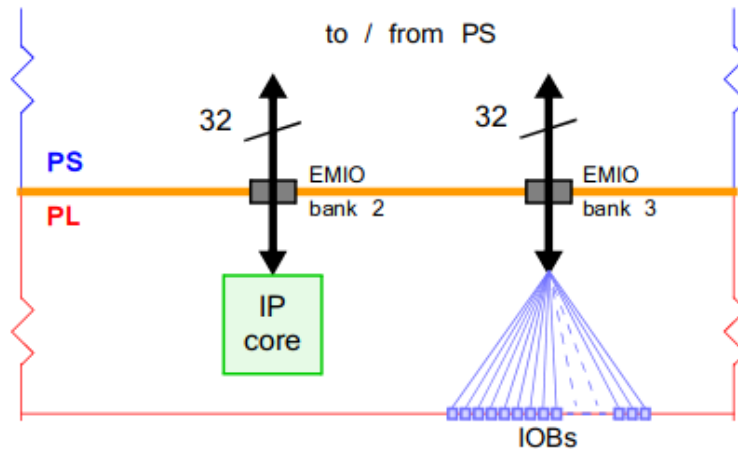
AXI: Primary Interface between the PS and PL

- AXI4 interconnect is a switch which manages and directs traffic between attached AXI interfaces.
- AXI4 interface is a point-to-point connection for passing data, addresses, and hand-shaking signals between master system such as the General Purpose Interf
- General Purpose AXI
- Accelerator Coherency Port
- High Performance Ports



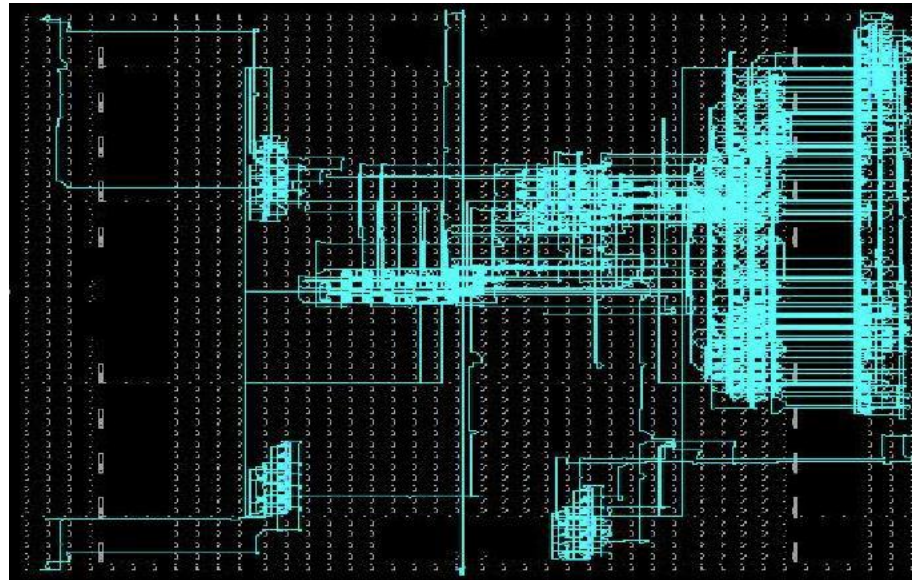
Primary Interface between the PS and PL: EMIO

- Connections from the PS can be routed through the PL to external interfaces with the Extended MIO (EMIO).
- Interfaces routed by the EMIO are usually connected directly to the external pins of the PL, as specified by the constraints file (.xdc), with 64 inputs, and 64 outputs with output enables or to use the EMIO to interface the PS with a peripheral block in the PL



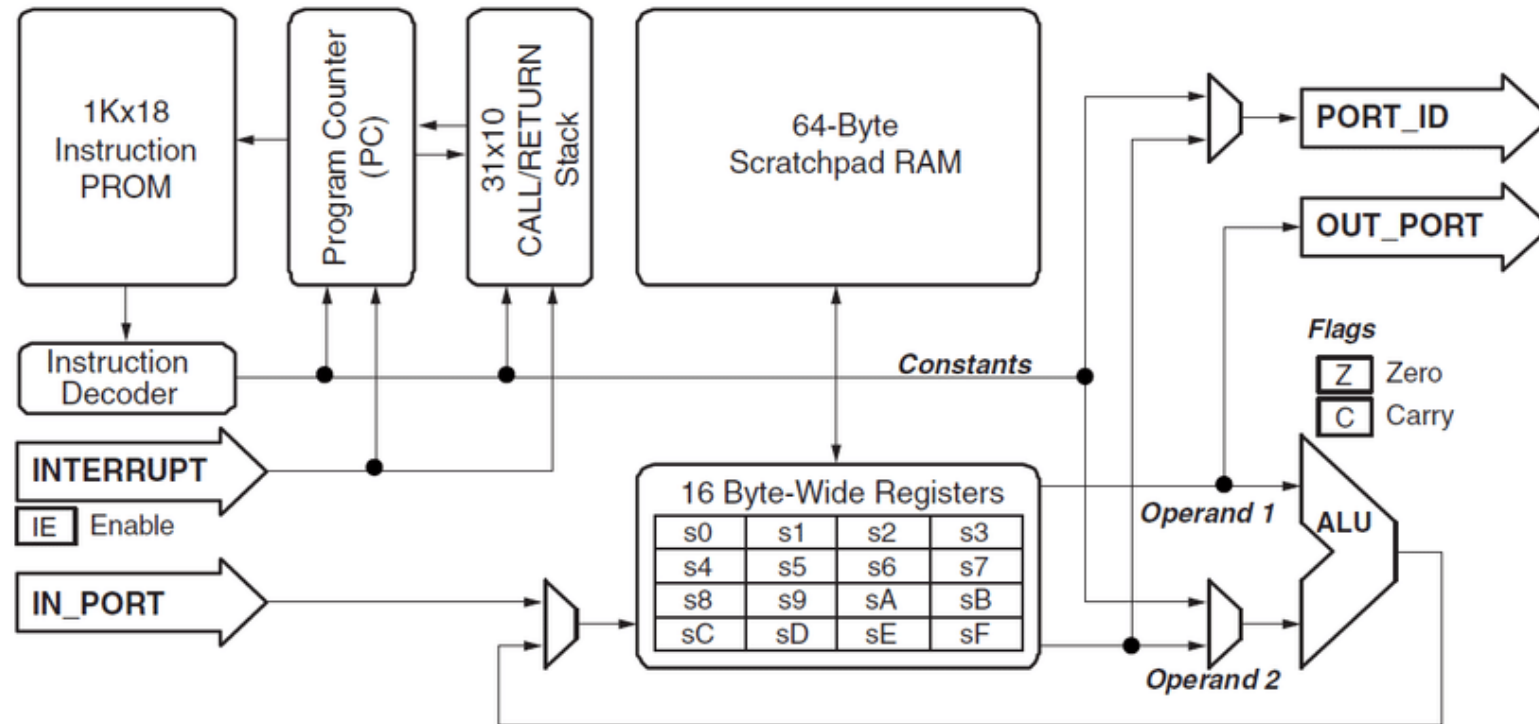
Embedded System

- The programmable logic (PL) of Zynq is equivalent to that of an FPGA. The Zynq XC7Z010 device of the Zybo board corresponds to the fabric of an Artix 7 FPGAs.
- The Zynq architecture includes an ARM Cortex A9 hard processor, but it is also possible to build soft core processors using the PL.



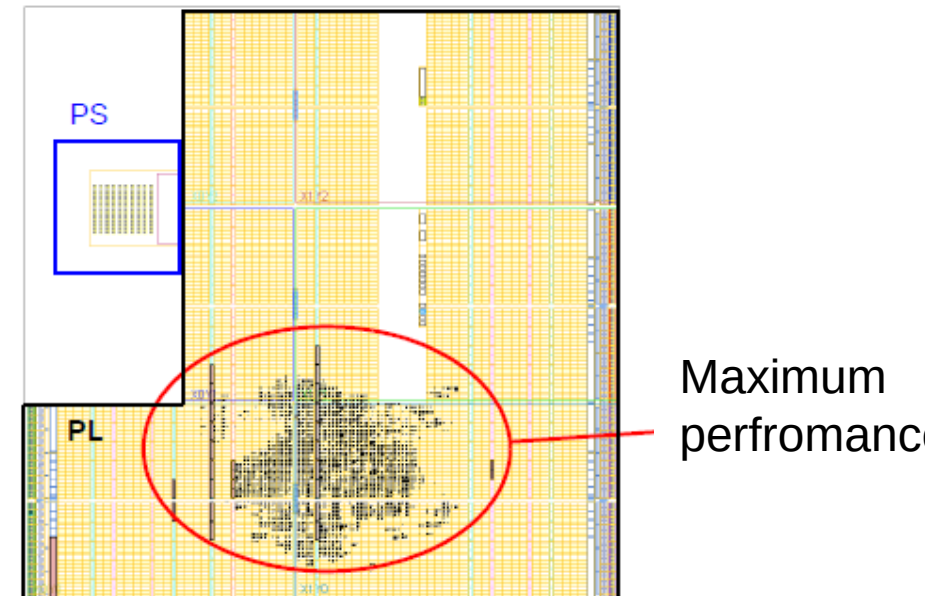
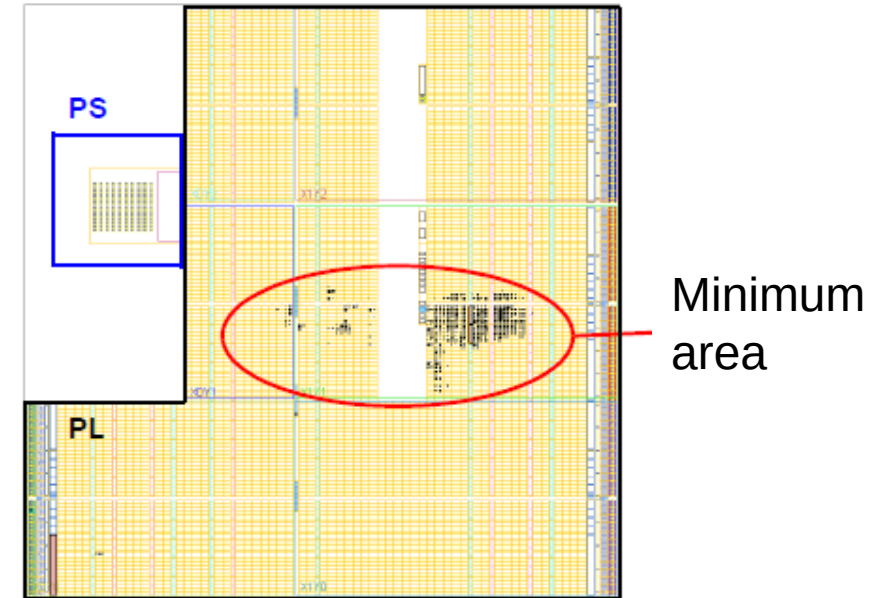
Soft core Example

- The Xilinx PicoBlaze is an 8 bit soft core microcontroller for low level functionality.
- The Xilinx MicroBlaze is a 32-bit soft core processor with a flexible configuration.



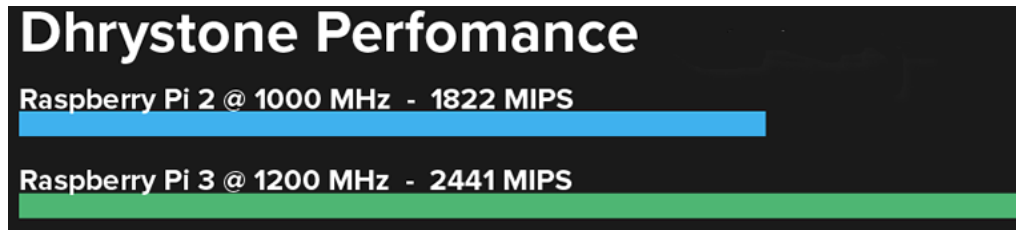
Soft core Example

- Customized to optimize for operating frequency, performance or area
- This is done very easily in Vivado using a configuration wizard(GUI).



Performance metrics for processors include:

- ***DMIPs (Dhrystone Millions of Instructions Per second)*** expresses the number of operations achieved per second undertaken by the processor when running the *Dhrystone* standard test application.



- ***CoreMark score*** is a numerical 'score' for processor performance, which can be directly compared with the scores of other processors based on tasks more typical of embedded processor usage.

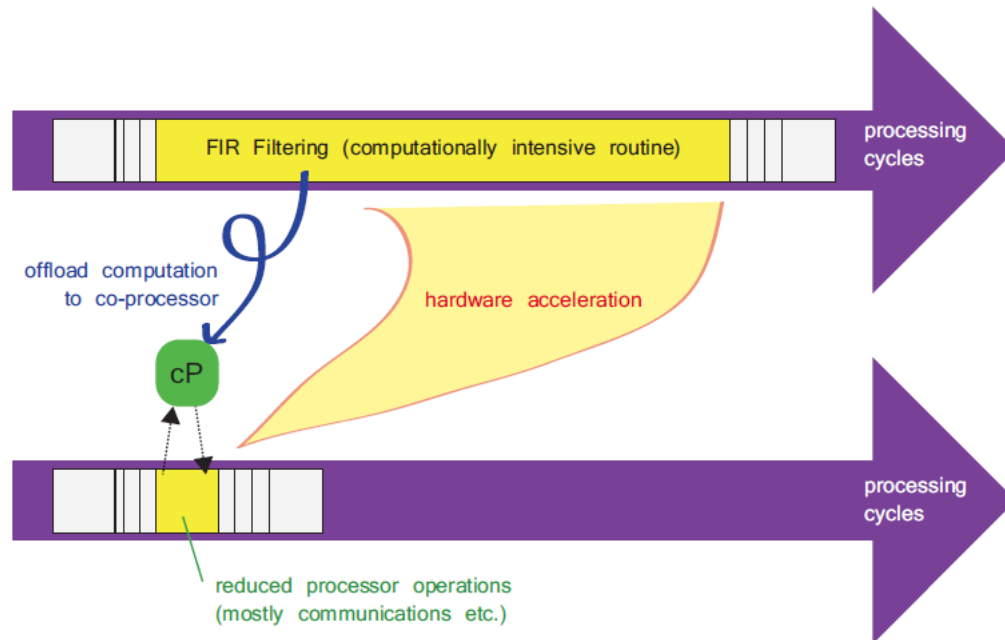
MicroBlaze and ARM Cortex A9 comparison

Processor Type	Configuration	Processing Performance (DMIPs) ^a
MicroBlaze	Area optimised (3-stage pipeline)	196
	Performance optimised with branch optimisations (5-stage pipeline)	228
	Performance optimised without branch optimisations (5-stage pipeline)	259
ARM Cortex-A9	1GHz; both cores combined; 2500 per core	5000

Processor Type	Configuration	Processing Performance (CoreMark) ^a
MicroBlaze	125MHz; 5-stage pipeline (Virtex-5)	238
ARM Cortex-A9	1GHz; both cores combined	5927
ARM Cortex-A9	800MHz ; both cores combined	4737

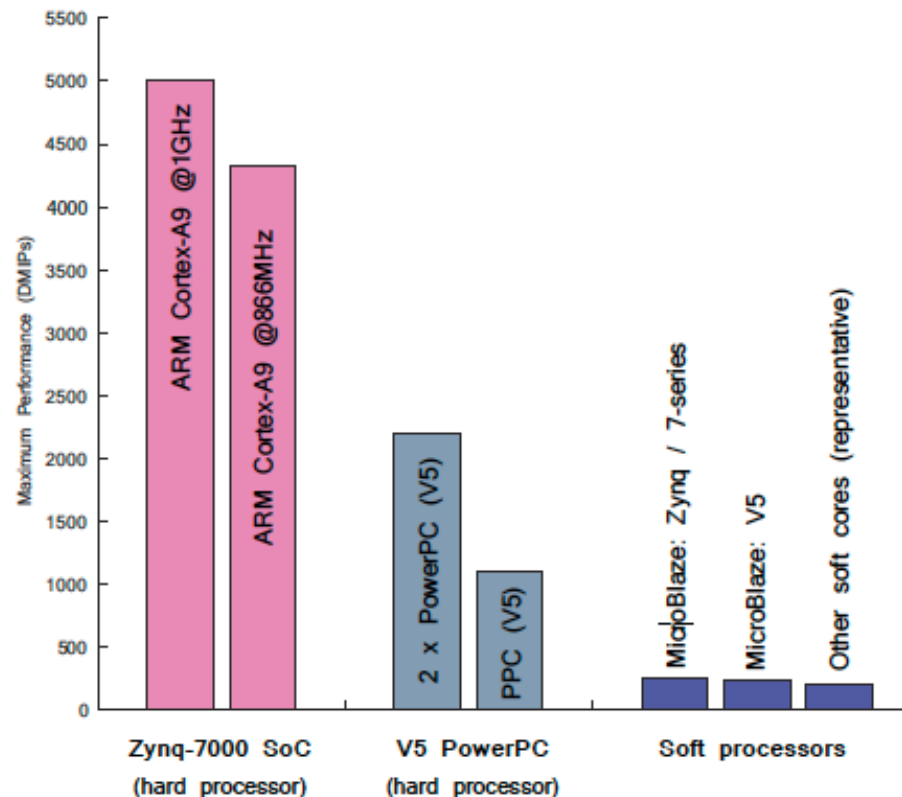
MicroBlaze and ARM Cortex A9 comparison

- The FPGA fabric of Zynq permits a high-speed, fully parallel version of an intensive processing operation to be *off-loaded* from the processor into hardware.
- This greatly reduces the overall execution time.



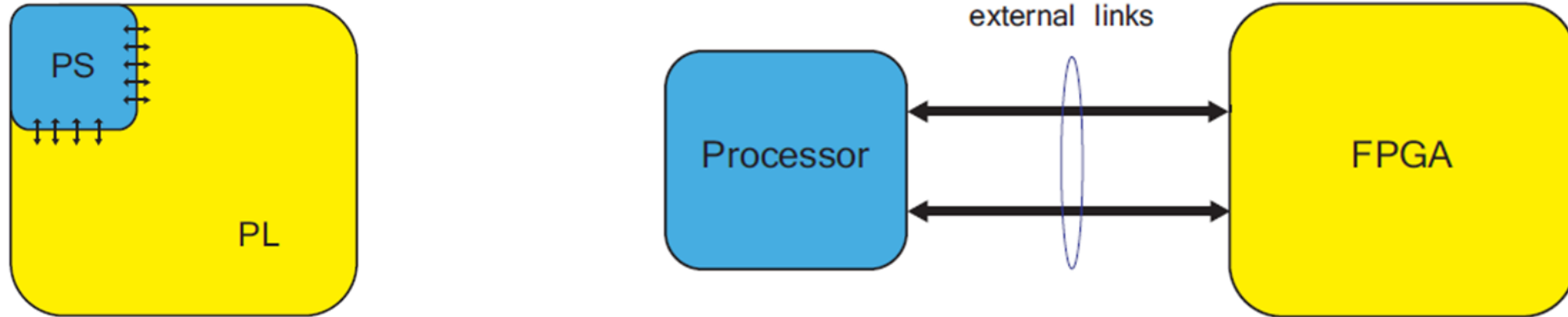
MicroBlaze and ARM Cortex A9 comparison

- The left and central groups represent hard processors, while the group on the right corresponds to a selection of soft processors.
- The Zynq-based ARM processor (leftmost group) operates at up to 866MHz (in the Artix-based devices) or up to 1GHz (in the Kintex-based devices).



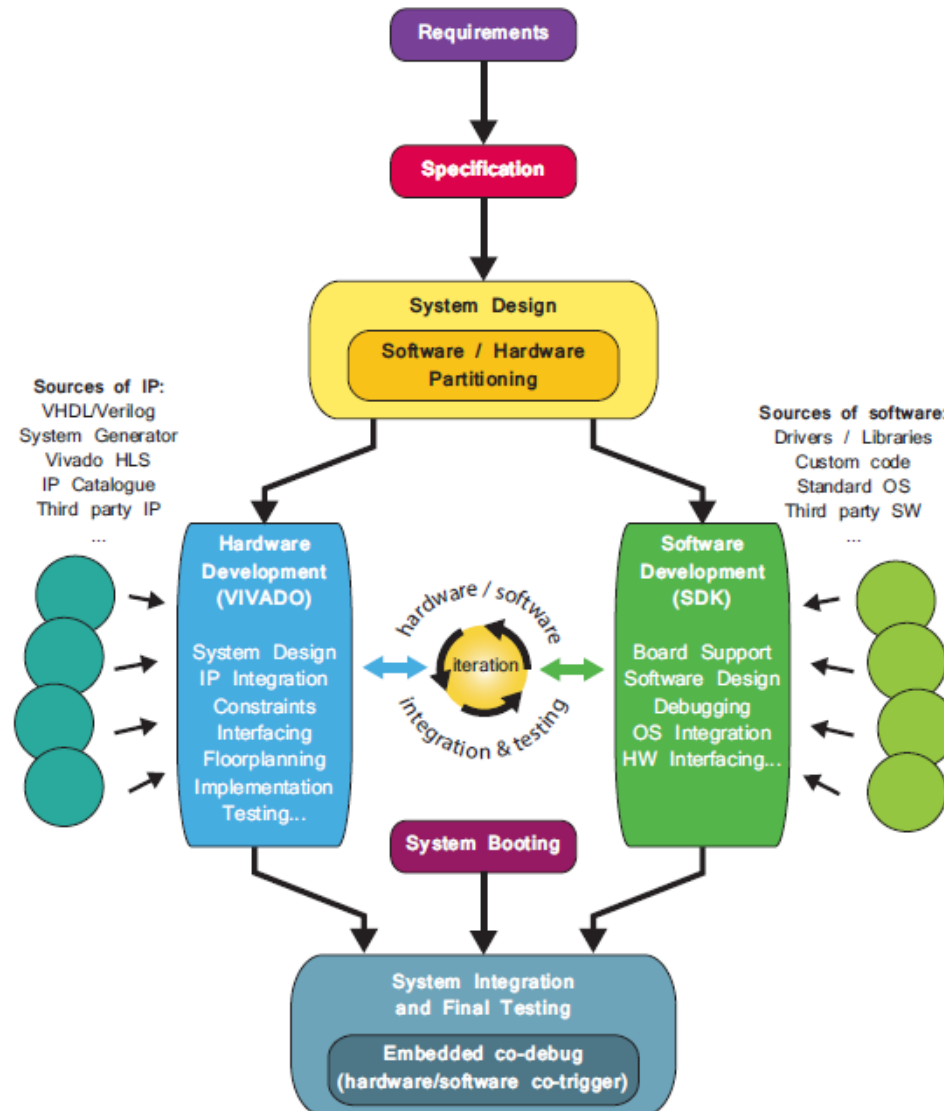
MicroBlaze and ARM Cortex A9 comparison

- The tight coupling between the PS and PL components of the Zynq has a low communication overhead compared to that of a discrete processor and FPGA



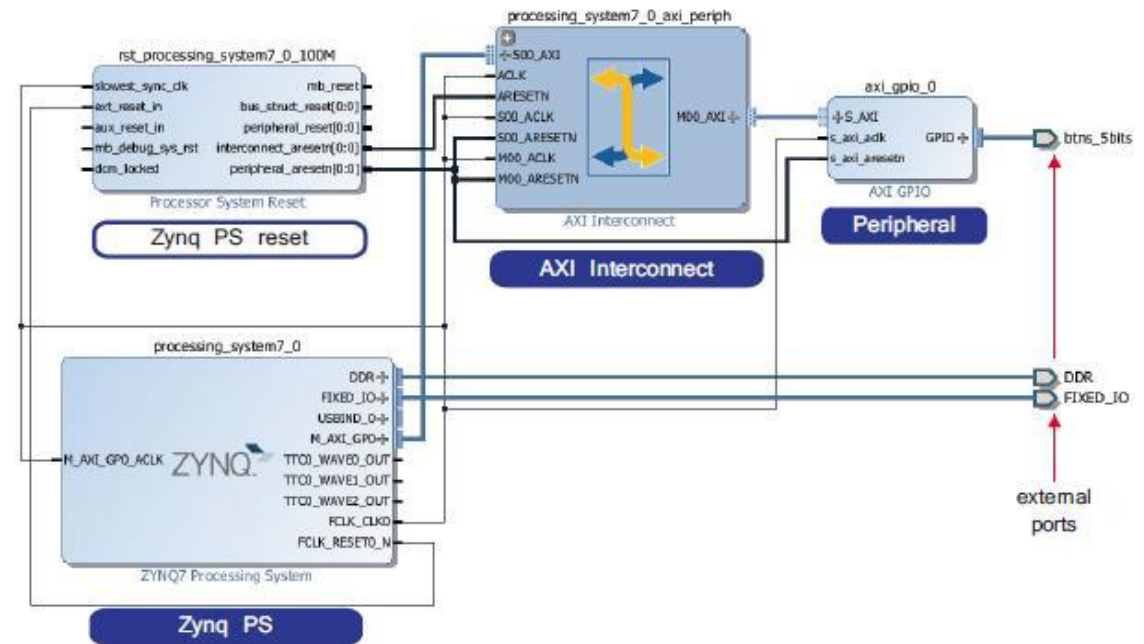
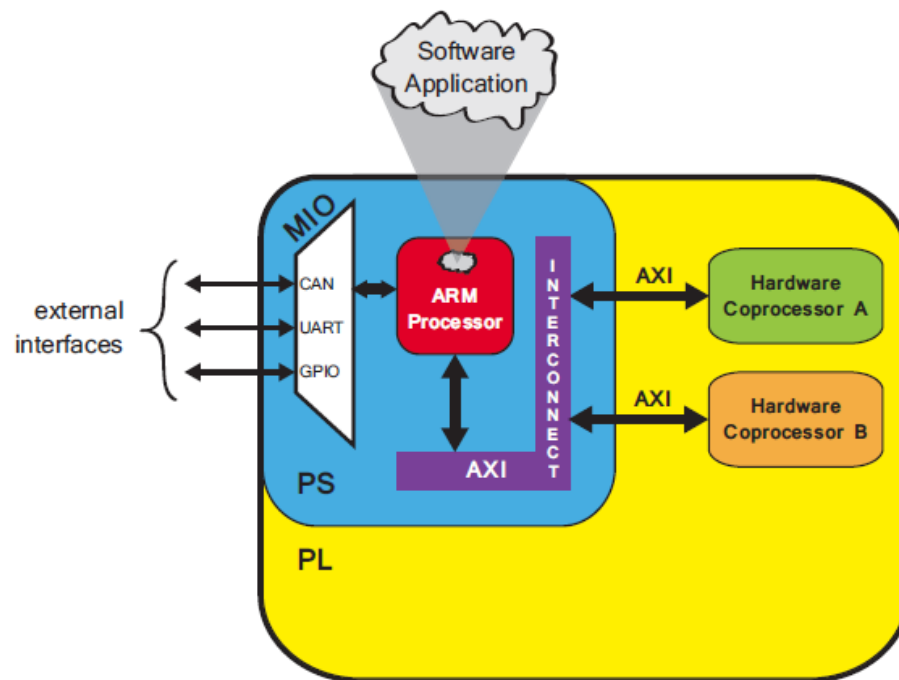
- Zynq provides a single-chip replacement for the effective processor-FPGA configuration.
- The external links expend more power than the local connections between the PS and PL in the Zynq which contributes to an overall power.

Designing with Zynq



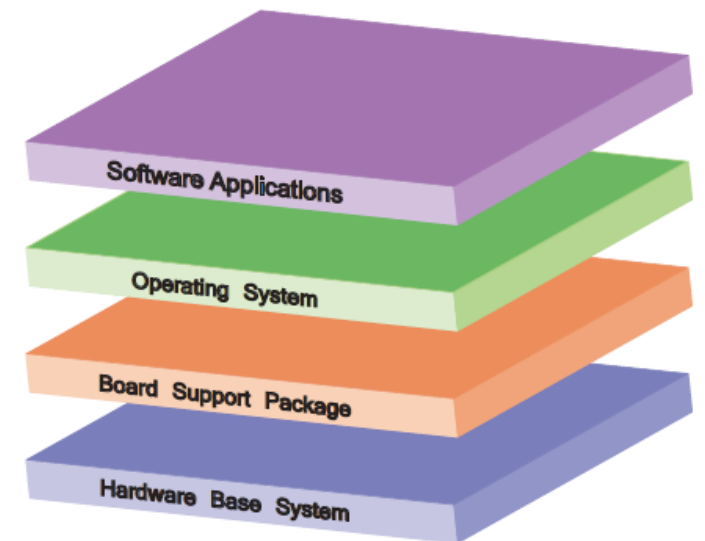
Designing with Zynq

- Development of the hardware system involves designing the peripheral blocks and other logic to be implemented in the PL, creating the appropriate connections between these blocks and the PS, and suitably configuring the PS.



Designing with Zynq

- The software system can be thought of as a stack, or set of layers, which is built upon the Hardware Base System.
- The layer directly above the Hardware Base System is the Board Support Package (BSP) which is a set of low level drivers and functions that are used by the next layer up (the Operating System).
- The Operating System communicates with the hardware down through the software stack.



Example FPGA-accelerated Addition

- Example FPGA-accelerated program

- PS side: host

```
std::vector<int, aligned_allocator<int> > source_in1(DATA_SIZE);  
std::vector<int, aligned_allocator<int> > source_in2(DATA_SIZE);  
std::vector<int, aligned_allocator<int> > source_hw_results(DATA_SIZE);  
std::vector<int, aligned_allocator<int> > source_sw_results(DATA_SIZE);
```

```
auto devices = xcl::get_xil_devices();  
// read_binary_file() is a utility API which will load the binaryFile  
// and will return the pointer to file buffer.  
auto fileBuf = xcl::read_binary_file(binaryFile);  
cl::Program::Binaries bins{{fileBuf.data(), fileBuf.size()}};  
bool valid_device = false;  
for (unsigned int i = 0; i < devices.size(); i++) {  
    auto device = devices[i];  
    // Creating Context and Command Queue for selected Device  
    OCL_CHECK(err, context = cl::Context(device, nullptr, nullptr, nullptr, &err));  
    OCL_CHECK(err, q = cl::CommandQueue(context, device, CL_QUEUE_PROFILING_ENABLE, &err));  
    std::cout << "Trying to program device[" << i << "]: " << device.getInfo<CL_DEVICE_NAME>() << std::endl;  
    cl::Program program(context, {device}, bins, nullptr, &err);  
    if (err != CL_SUCCESS) {  
        std::cout << "Failed to program device[" << i << "] with xclbin file!\n";  
    } else {  
        std::cout << "Device[" << i << "]: program successful!\n";  
        OCL_CHECK(err, krnl_vector_add = cl::Kernel(program, "vadd", &err));  
        valid_device = true;  
        break; // we break because we found a valid device  
    }  
}
```

Refer: https://github.com/Xilinx/Vitis_Accel_Examples/tree/master/hello_world

Example FPGA-accelerated Addition

- Example FPGA-accelerated program
- PS side:

```
OCL_CHECK(err, cl::Buffer buffer_in1(context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY, vector_size_bytes,  
                                         source_in1.data(), &err));  
OCL_CHECK(err, cl::Buffer buffer_in2(context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY, vector_size_bytes,  
                                         source_in2.data(), &err));  
OCL_CHECK(err, cl::Buffer buffer_output(context, CL_MEM_USE_HOST_PTR | CL_MEM_WRITE_ONLY, vector_size_bytes,  
                                         source_hw_results.data(), &err));
```

```
int size = DATA_SIZE;  
OCL_CHECK(err, err = krnl_vector_add.setArg(0, buffer_in1));  
OCL_CHECK(err, err = krnl_vector_add.setArg(1, buffer_in2));  
OCL_CHECK(err, err = krnl_vector_add.setArg(2, buffer_output));  
OCL_CHECK(err, err = krnl_vector_add.setArg(3, size));
```

```
OCL_CHECK(err, err = q.enqueueTask(krnl_vector_add));
```

```
// Copy Result from Device Global Memory to Host Local Memory  
OCL_CHECK(err, err = q.enqueueMigrateMemObjects({buffer_output}, CL_MIGRATE_MEM_OBJECT_HOST));  
q.finish();  
// OPENCL HOST CODE AREA END
```

Refer: https://github.com/Xilinx/Vitis_Accel_Examples/tree/master/hello_world

Example FPGA-accelerated Addition

- Example FPGA-accelerated program
- PL side: vadd.cpp

```
static void compute_add(hls::stream<hls::vector<uint32_t, NUM_WORDS> >& in1_stream,  
                        hls::stream<hls::vector<uint32_t, NUM_WORDS> >& in2_stream,  
                        hls::stream<hls::vector<uint32_t, NUM_WORDS> >& out_stream,  
                        int vSize) {  
    // The kernel is operating with vector of NUM_WORDS integers. The + operator performs  
    // an element-wise add, resulting in NUM_WORDS parallel additions.  
    execute:  
        for (int i = 0; i < vSize; i++) {  
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size  
            out_stream << (in1_stream.read() + in2_stream.read());  
        }  
    }  
    #pragma HLS dataflow  
  
    load_input(in1, in1_stream, vSize);  
    load_input(in2, in2_stream, vSize);  
    compute_add(in1_stream, in2_stream, out_stream, vSize);  
    store_result(out, out_stream, vSize);  
}
```

Refer: https://github.com/Xilinx/Vitis_Accel_Examples/tree/master/hello_world